

1. Interfaces

Java Collections Framework의 **핵심 컬렉션 인터페이스(Core Collection Interfaces)** 를 정리한 글이다. 각 인터페이스의 계약(contract)과 공식 문서가 강조하는 주의사항을 중심으로 다룬다.

핵심 컬렉션 인터페이스는 Java Collections Framework의 근간이며, 컬렉션을 그 표현 방식의 세부 구현과 무관하게 조작할 수 있게 한다. 이때 문서가 강조하는 전제가 있다.

i. 계층은 두 개의 별개 트리로 구성된다

하나는 **Collection** 을 루트로 하는 트리이고, 다른 하나는 **Map** 트리이다. 이때 공식 문서에서는 **Map 이 진정한 Collection 이 아니라고** 하며, **Map 은 Collection 을 상속하지 않는다.**

다만 Map 이 프레임워크에서 배제되는 것은 아니다. **keySet** , **values** , **entrySet** 같은 **컬렉션 뷰(Collection View)** 를 통해 Map 도 컬렉션처럼 다룰 수 있으며, 이 뷰가 두 트리를 이어주는 통로 역할을 한다.

ii. 모든 핵심 인터페이스는 제네릭이다

```
// 선언
public interface Collection<E> {
    // something
}
```

컬렉션 인스턴스를 선언할 때 담을 객체의 타입을 지정하면 컴파일 시점에 타입 검증이 이루어져 런타임 오류를 줄일 수 있다. 타입 인자를 생략한 로(raw) 타입을 사용하면 이 검증을 잃게 되므로, 항상 타입 인자를 명시하는 것이 원칙이다.

```
// Java 7 이상에서는 다이아몬드 연산자로 우변의 타입 인자를 생략할 수 있다.
List<String> list = new ArrayList<>();
```

iii. 선택적 연산(Optional Operations)

불변, 고정 크기, 추가 전용 등 변형마다 별도의 인터페이스를 두면 인터페이스 수가 관리 불가능해진다. 그래서 프레임워크는 각 인터페이스의 **수정 연산을 선택적(optional)으로 규정한다.**

특정 구현체는 일부 연산을 지원하지 않을 수 있으며, 지원하지 않는 연산을 호출하면 **UnsupportedOperationException** 이 발생한다. 따라서 각 구현체가 어떤 선택적 연산을 지원하는지는 해당 구현체의 문서에 명시해야 한다.

```
List<String> fixed = Arrays.asList("a", "b", "c");
fixed.add("d"); // UnsupportedOperationException (고정 크기 리스트)
```

```
List<String> immutable = List.of("a", "b", "c");
immutable.set(0, "z"); // UnsupportedOperationException (불변 리스트)
```

iv. 핵심 인터페이스

Collection, Set, List, Queue, Deque, Map 그리고 정렬 계약을 추가한 SortedSet, SortedMap

Set 은 Collection 의 특수한 종류이고, SortedSet 은 Set 의 특수한 종류이다. Map 트리도 Map → SortedMap 으로 이와 유사한 관계를 가진다.

1. Collection 트리

1.1 The Collection Interface

Collection 은 원소들의 그룹을 나타내는 계층의 루트 인터페이스이다. 문서는 이를 **최대한의 일반성이 필요할 때 컬렉션을 주고받고 조작하는 데 쓰이는 최소 공통 분모**로 규정한다. 자바 플랫폼은 이 인터페이스의 직접 구현체를 제공하지 않고, Set, List 같은 **더 구체적인 하위 인터페이스의 구현체만** 제공한다.

관례상 모든 범용 구현체는 Collection 을 인자로 받는 **변환 생성자(conversion constructor)** 를 제공한다. 이를 통해 컬렉션의 구현 타입을 손쉽게 바꾸거나 복사할 수 있다.

```
Collection<String> c = ... ;

List<String> list = new ArrayList<>(c); // 어떤 컬렉션이든 List 로 변환
Set<String> set = new HashSet<>(c); // 중복을 제거한 Set 으로 변환
```

연산은 기본 연산, 컬렉션 전체 대상 연산, 배열 연산으로 나뉜다.

분류	연산
기본 연산	size, isEmpty, contains, add, remove, iterator
컬렉션 전체 대상(벌크) 연산	containsAll, addAll, removeAll, retainAll, clear
배열 연산	toArray

Java 8 이상부터는 stream, parallelStream, removeIf, forEach 메서드도 함께 노출된다.

add 는 중복을 허용하는 컬렉션과 그렇지 않은 컬렉션 **모두**에 대해 성립하도록 정의된다. 호출이 완료된 후 해당 원소가 컬렉션에 포함되어 있음을 보장하며, 호출 결과 컬렉션이 변경되면 **true** 를 반환한다. 따라서 중복을 허용하지 않는 Set 에 이미 존재하는 원소를 넣으면 예외가 아니라 **false** 가 반환된다.

remove 는 지정된 원소의 **단일 인스턴스**를 제거하며, 변경이 발생하면 **true** 를 반환한다.

컬렉션 순회에는 세 가지 방식이 있다.

- 집계 연산(Aggregate Operations), `for-each`, `Iterator`

집계 연산 : Aggregate Operations

JDK 8 이상에서 선호되는 방식으로, 스트림을 얻어 연산을 수행한다.

```
myCollection.stream()
    .filter(e → e.getColor() == Color.RED)
    .forEach(e → System.out.println(e.getName()));
```

기존의 벌크 연산 중 `addAll`, `removeAll`, `retainAll`, `clear` 는 모두 변형적(destructive)이어서 대상 컬렉션 자체를 수정한다. (`containsAll` 은 조회만 하므로 예외이다.)

반면 JDK 8 의 집계 연산은 원본 컬렉션을 수정하지 않는다. 특히 병렬 스트림에서 문제를 일으키지 않도록, 스트림 파이프라인 내부에서는 원본을 변형하는 동작을 피해야 한다.

```
// 병렬 스트림도 동일한 파이프라인으로 처리 가능하다.
myCollection.parallelStream()
    .filter(e → e.getColor() == Color.RED)
    .collect(Collectors.toList());
```

for-each

```
for (Object o : collection) {
    System.out.println(o);
}
```

내부적으로는 `Iterator` 를 사용하지만, 반복자 변수를 직접 노출하지 않아 코드가 간결하다. 대신 순회 도중 원소를 제거할 수 없다.

Iterator

`Iterator` 인터페이스는 `hasNext`, `next` 그리고 선택적 연산인 `remove` 로 구성된다.

```
public interface Iterator<E> {
    boolean hasNext();
    E next();
    void remove(); // optional
}
```

문서에서는 `Iterator.remove` 가 순회 도중 컬렉션을 수정하는 유일하게 안전한 방법이라고 한다. 다른 방식으로 컬렉션을 수정하면 동작이 명세되지 않으며, 대부분의 구현체는 이를 감지해 `ConcurrentModificationException` 을 던진다.

```
// 잘못된 예 : 순회 중 컬렉션을 직접 수정
for (String s : list) {
    if (s.isEmpty()) list.remove(s); // ConcurrentModificationException
}
```

다음과 같은 경우에 `for-each` 대신 `Iterator` 를 사용한다.

- 현재 원소를 제거해야 할 때
- 여러 컬렉션을 병렬로 순회할 때

```
static void filter(Collection<?> c) {
    for (Iterator<?> it = c.iterator(); it.hasNext(); ) {
        if (!cond(it.next())) it.remove();
    }
}
```

Java 8 이상에서는 같은 동작을 `c.removeIf(e → !cond(e));` 로 간결하게 표현할 수 있다.

1.2 The Set Interface

`Set` 은 중복 원소를 포함할 수 없는 `Collection` 으로, 수학적 집합(set) 추상을 모델링한다. `Collection` 으로부터 상속한 메서드만 포함하되 **중복 금지 제약**을 추가하며, `equals` 와 `hashCode` 의 동작에 더 강한 계약을 부과해 구현 타입이 달라도 의미 있게 비교할 수 있게 한다. 즉, 두 `Set` 인스턴스는 동일한 원소를 담고 있으면 서로 같다.

범용 구현체는 세 가지다.

구현체	내부 구조	특징
HashSet	해시 테이블	성능이 가장 우수하지만 반복 순서를 보장하지 않는다
TreeSet	레드-블랙 트리	값에 따라 정렬되지만 HashSet 보다 상당히 느리다
LinkedHashSet	연결 리스트가 관통하는 해시 테이블	삽입 순서를 유지하며, HashSet 보다 약간 높은 비용으로 예측 불가능한 순서 문제를 해소한다

```
Collection<Type> noDups1 = new HashSet<>();
Collection<Type> noDups2 = new LinkedHashSet<>();
```

컬렉션은 항상 구현 타입이 아니라 **인터페이스 타입으로 참조해야 한다**. 그래야 생성자만 바꿔 구현을 교체할 수 있는 유연성이 확보된다.

`HashSet` 을 `TreeSet` 으로 바꾸는 한 줄 변경만으로도 출력이 정렬되게 할 수 있다.

벌크 연산은 집합 대수와 정확히 대응한다.

연산	의미
<code>s1.containsAll(s2)</code>	s2 가 s1 의 부분 집합인지 여부
<code>s1.addAll(s2)</code>	합집합 ($s1 \leftarrow s1 \cup s2$)
<code>s1.retainAll(s2)</code>	교집합 ($s1 \leftarrow s1 \cap s2$)
<code>s1.removeAll(s2)</code>	차집합 ($s1 \leftarrow s1 - s2$)

이 연산들은 **호출 대상인 s1 을 직접 변형한다는 점**에 주의해야 한다. 원본을 보존하려면 복사본을 만들어 사용한다.

```
Set<Type> union = new HashSet<>(s1);
union.addAll(s2);

Set<Type> intersection = new HashSet<>(s1);
intersection.retainAll(s2);
```

1.3 The List Interface

List 는 순서가 있는 **Collection** (시퀀스)이며 중복을 허용한다. **Collection** 에서 상속한 연산 외에 네 부류의 연산을 추가한다.

분류	연산
위치 기반 접근	<code>get</code> , <code>set</code> , <code>add</code> , <code>addAll</code> , <code>remove</code>
검색	<code>indexOf</code> , <code>lastIndexOf</code>
반복	<code>listIterator</code>
범위 뷰	<code>subList</code>

범용 구현체는 두 가지로 나뉜다. 일반적으로 성능이 더 나은 **ArrayList** 와, 특정 상황에서 유리한 **LinkedList**이다. **ArrayList** 는 인덱스 기반 접근이 $O(1)$ 로 빠르고, **LinkedList** 는 리스트의 앞 쪽 삽입·삭제가 잦거나 **Deque** 로 쓸 때 유리하다.

마찬가지로 **List** 도 `equals` , `hashCode` 에 더 강한 요구를 부과하며, 두 리스트는 **동일한 원소를 동일한 순서로** 담을 때 같다.

Collection 에서 상속한 연산 중 의미가 좁혀진 것들이 있다.

- `remove` 는 항상 **첫 번째로 일치하는 원소**를 제거한다.
- `add` , `addAll` 은 리스트의 **끝에 덧붙인다**.

```
list.remove(1); // 인덱스 1의 원소 제거 (위치 기반)
list.remove(Integer.valueOf(1)); // 값이 1인 첫 원소 제거 (객체 기반)
```

`List<Integer>` 에서 `remove(int)` 와 `remove(Object)` 가 오버로딩으로 갈리므로, 의도한 쪽을 명확히 해야 한다.

ListIterator

`ListIterator` 는 리스트가 제공하는 더 풍부한 반복자로 **양방향 순회, 순회 중 수정, 현재 위치 조회**를 지원한다. `Iterator` 로부터 상속한 `hasNext`, `next`, `remove` 에 더해 그 역방향인 `hasPrevious`, `previous`, 위치를 알려주는 `nextIndex`, `previousIndex`, 그리고 수정 연산인 `set`, `add` 를 제공한다.

여기서 **커서(cursor)** 개념이 들어간다. 커서는 항상 두 원소 **사**이에 위치하며, 길이 n 의 리스트에는 0부터 n 까지 n+1 개의 유효한 커서 위치가 존재한다.

	0	1	2	3
커서 →	e0	e1	e2	

- `next` 는 커서를 앞으로 넘기며 그 위치의 원소를 반환한다.
- `previous` 는 커서를 뒤로 넘기며 그 위치의 원소를 반환한다.
- `set` 은 마지막으로 반환된 원소를 덮어쓴다.
- `add` 는 현재 커서 위치 바로 앞에 새 원소를 삽입한다. 따라서 `add` 직후의 `previous` 는 방금 넣은 원소를, `next` 는 영향을 받지 않는다.

```
// 리스트를 역방향으로 순회
for (ListIterator<Type> it = list.listIterator(list.size());
it.hasPrevious(); ) {
    Type t = it.previous();
    // ...
}

// 특정 값을 모두 치환
for (ListIterator<String> it = list.listIterator(); it.hasNext(); ) {
    if (it.next().equals("old")) it.set("new");
}
```

범위 뷰 : subList

범위 뷰 `subList(fromIndex, toIndex)` 는 해당 부분 리스트의 뷰를 반환한다. **뷰(view)** 라는 용어대로 반환된 리스트는 원본에 의해 뒷받침되므로 변경이 서로 반영된다. 이를 통해 별도의 범위 연산이 없어도 다음과 같은 관용구가 가능하다.

```
list.subList(fromIndex, toIndex).clear();
```

```
// fromIndex : 포함 (inclusive)
// toIndex   : 제외 (exclusive)
```

```
// 범위 안에서의 검색도 뷰로 처리할 수 있다.
int i = list.subList(fromIndex, toIndex).indexOf(o);
```

문서에서는 `subList` 사용에 주의해야 한다고 한다. 뷰를 만든 뒤 원본 리스트를 직접 수정하면 그 뷰의 의미가 정의되지 않으므로, `subList` 로 얻은 인스턴스는 일련의 범위 연산을 수행하는 일시적인 객체로만 사용할 것을 권고한다.

1.4 The Queue Interface

`Queue` 는 처리에 앞서 원소를 보관하는 컬렉션으로, 기본 컬렉션 연산 외에 삽입·제거·검사 연산을 추가한다. 핵심 구조는 **각 연산이 두 가지 형태로 존재한다**는 점이다. 하나는 실패 시 예외를 던지고, 다른 하나는 특수한 값(`null` 또는 `false`)을 반환한다.

연산	예외를 던지는 형태	특수 값을 반환하는 형태
삽입(Insert)	<code>add(e)</code>	<code>offer(e) → false</code>
제거(Remove)	<code>remove()</code>	<code>poll() → null</code>
검사(Examine)	<code>element()</code>	<code>peek() → null</code>

큐는 대체로 FIFO 순서를 따르지만 반드시 그런 것은 아니다. 예외적으로 우선순위 큐(`PriorityQueue`)는 원소의 값에 따라 정렬한다. 어떤 순서를 쓰든 큐의 **head** 는 `remove` 나 `poll` 호출 시 제거될 원소이며, 모든 구현체는 자신의 순서 속성을 명세해야 한다.

`add` 와 `offer` 의 차이는 **용량 제한이 있는 큐**에서 드러난다. 전자는 용량 위반 시 `IllegalStateException` 을 던지며, 후자는 `false` 를 반환한다.

`remove / poll` , `element / peek` 은 **큐가 비었을 때만** 동작이 걸린다. 전자는 `NoSuchElementException` 을 던지고, 후자는 `null` 을 반환한다.

구현체는 일반적으로 `null` 삽입을 허용하지 않는다. `LinkedList` 는 예외적으로 허용하지만, `poll / peek` 이 `null` 을 특수 반환 값으로 사용하므로 이를 활용하지 않을 것을 권장한다.

```
Queue<Integer> queue = new LinkedList<>();
queue.offer(1);
queue.offer(2);

System.out.println(queue.peek()); // 1 (제거하지 않고 확인)
System.out.println(queue.poll()); // 1 (제거하며 반환)
System.out.println(queue.poll()); // 2
System.out.println(queue.poll()); // null (비어 있음)
```

1.5 The Deque Interface

덱(Deque, double ended queue)은 양 끝점 모두에서 원소의 삽입과 제거를 지원하는 선형 컬렉션이다. 덱은 스택과 큐를 동시에 구현하므로, 스택이나 큐보다 풍부한 추상 데이터 타입을 가진다. 구현체로는 **ArrayDeque**, **LinkedList** 가 있다.

연산	첫 원소 (예외 / 특수 값)	마지막 원소 (예외 / 특수 값)
삽입	<code>addFirst(e)</code> / <code>offerFirst(e)</code>	<code>addLast(e)</code> / <code>offerLast(e)</code>
제거	<code>removeFirst()</code> / <code>pollFirst()</code>	<code>removeLast()</code> / <code>pollLast()</code>
조회	<code>getFirst()</code> / <code>peekFirst()</code>	<code>getLast()</code> / <code>peekLast()</code>

용량 제한이 있을 때는 `offerFirst`, `offerLast` 를 선호한다. 제거 및 조회 계열에서 `pollFirst`, `pollLast`, `peekFirst`, `peekLast` 는 덱이 비었을 때 `null` 을 반환한다. 반면 `removeFirst`, `removeLast`, `getFirst`, `getLast` 는 비어 있을 때 예외를 던진다.

Deque 는 `push`, `pop`, `peek` 같은 스택 연산도 제공하므로, 레거시 클래스인 **Stack** 대신 **ArrayDeque** 를 사용하는 것이 권장된다.

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); // addFirst 와 동일
stack.push(2);
System.out.println(stack.pop()); // 2 (removeFirst 와 동일, LIFO)
```

2. Map 트리

2.1 The Map Interface

Map 은 키를 값에 대응시키는 객체로, 수학적 함수 추상을 모델링한다. 중복 키를 가질 수 없으며, 각 키는 **최대 하나의 값**에 대응한다.

분류	연산
기본 연산	<code>put</code> , <code>get</code> , <code>remove</code> , <code>containsKey</code> , <code>containsValue</code> , <code>size</code> , <code>isEmpty</code>
벌크 연산	<code>putAll</code> , <code>clear</code>
컬렉션 뷰	<code>keySet</code> , <code>entrySet</code> , <code>values</code>

범용 구현체로는 **HashMap**, **TreeMap**, **LinkedHashMap** 이 있으며, 동작과 성능은 각 `HashSet`, `TreeSet`, `LinkedHashSet` 에 대응된다.


```
Map<String, Integer> map = new HashMap<>();
```

`Set`, `List` 와 마찬가지로 두 맵은 **동일한 Key-Value 대응을 나타내면 같다**. 관례상 모든 범용 맵 구현체는 맵을 인자로 받는 표준 변환 생성자를 제공한다.

```
Map<String, Integer> copy = new TreeMap<>(map); // 키 순서로 정렬된 복사본
```

컬렉션 뷰

컬렉션 뷰는 맵을 순회하는 유일한 수단이다.

- `keySet` : 키들의 **Set**
- `values` : 값들의 **Collection** (값은 중복될 수 있으므로 `Set` 이 아니다)
- `entrySet` : key-value 쌍의 **Set** 이며, 원소 타입은 중첩 인터페이스 `Map.Entry` 이다

```
for (Map.Entry<KeyType, ValueType> e : m.entrySet()) {  
    System.out.println(e.getKey() + " : " + e.getValue());  
}
```

문서는 "뷰를 호출할 때마다 새 컬렉션 인스턴스가 생성되어 느릴 것"이라는 우려에 대해, `java.util` 의 모든 맵 구현체가 주어진 뷰에 대해 **동일한 객체를 반환**하므로 문제가 없다고 설명한다.

순회 중 수정과 관련해서는 다음 규칙이 적용된다.

- 세 뷰 모두 `Iterator.remove` 로 맵의 항목을 제거할 수 있다.
- `entrySet` 뷰에서는 `Map.Entry` 의 `setValue` 로 값을 변경할 수 있다.
- **이것이 순회 중 맵을 수정하는 유일한 방법**이며, 그 외의 방식으로 수정하면 동작이 명세되지 않는다.
- 컬렉션 뷰는 **어떤 경우에도 원소 추가를 지원하지 않는다**.

```
// 값이 0인 항목을 순회 중 제거  
for (Iterator<Map.Entry<String, Integer>> it = m.entrySet().iterator();  
it.hasNext(); ) {  
    Map.Entry<String, Integer> e = it.next();  
    if (e.getValue() == 0) it.remove();  
}
```

Java 8 이상에서

는 `getOrDefault`, `putIfAbsent`, `computeIfAbsent`, `merge`, `forEach` 등이 추가되어 반복적인 null 체크 코드를 줄일 수 있다.

```
// 단어 빈도수 세기  
map.merge(word, 1, Integer::sum);
```

2.2 Object Ordering

정렬 기준을 정하는 두 가지 메커니즘을 다룬다.

Comparable

`Comparable` 인터페이스는 클래스에 **자연 순서(natural ordering)**를 부여하여 해당 객체를 자동으로 정렬할 수 있게 한다. 단일 메서드 `compareTo(T o)`로 구성되며, 수신 객체가 인자보다 작으면 음의 정수, 같으면 0, 크면 양의 정수를 반환한다.

`Comparable`을 구현하지 않은 원소로 이루어진 리스트를 `Collections.sort`로 정렬하려 하면 `ClassCastException`이 발생한다.

`compareTo` 구현의 전형은 **가장 유효한(중요한) 부분부터 비교하여 차이가 나면 즉시 반환하고, 같으면 다음 부분으로 넘어가는** 방식이다.

```
public int compareTo(Name n) {
    int lastCmp = lastName.compareTo(n.lastName);
    return (lastCmp != 0 ? lastCmp : firstName.compareTo(n.firstName));
}
```

문서에서는 `Comparable`을 구현하는 클래스가 지켜야 할 **네 가지 제약**이 자연 순서를 **전순서(total order)**로 보장하며, 이를 위반하면 정렬 동작이 정의되지 않는다고 한다.

1. `sgn(x.compareTo(y)) == -sgn(y.compareTo(x))` — 대칭성
2. 추이성 : `x.compareTo(y) > 0 && y.compareTo(z) > 0` 이면 `x.compareTo(z) > 0`
3. `x.compareTo(y) == 0` 이면 모든 `z`에 대해 `sgn(x.compareTo(z)) == sgn(y.compareTo(z))`
4. (강력 권고) `(x.compareTo(y) == 0) == x.equals(y)` — **`equals`와의 일관성**

또한 예제 `Name` 클래스를 통해 다음을 제시한다.

- 불변(**immutable**)으로 설계할 것
- 생성자에서 **null** 검사를 할 것
- **`equals`**를 재정의하면 반드시 **`hashCode`**도 재정의할 것

Comparator

정렬 기준이 클래스의 자연 순서와 다르거나, 자연 순서가 없는 클래스를 정렬해야 할 때는 `Comparator`를 사용한다. `compare(T o1, T o2)`는 `compareTo`와 동일한 반환 규약을 따른다.

```
// 입사일 순으로 정렬하는 Comparator
Comparator<Employee> byHireDate = (e1, e2) ->
    e1.hireDate().compareTo(e2.hireDate());

Collections.sort(employees, byHireDate);
```

Java 8 이상에서는 조합형 팩터리 메서드를 사용하면 훨씬 간결하다.

```
employees.sort(  
    Comparator.comparing(Employee::hireDate)  
                .thenComparing(Employee::name)  
                .reversed()  
);
```

문서는 두 가지 함정을 특히 경고한다.

1. 결과를 부정(negate)하는 방식

역순 비교기를 만들 때 `compareTo` 의 결과에 단순히 마이너스를 붙이는 방식은 안전하지 않다. `compareTo` 가 `Integer.MIN_VALUE` 를 반환하면 이를 부정해도 오버플로로 인해 그대로 `Integer.MIN_VALUE` (음수)가 되기 때문이다. 따라서 **인자의 순서를 바꾸는 방식**을 사용해야 한다.

```
// 잘못된 방식  
public int compare(Employee e1, Employee e2) {  
    return -e1.compareTo(e2);  
}  
  
// 올바른 방식  
public int compare(Employee e1, Employee e2) {  
    return e2.compareTo(e1);  
}
```

같은 이유로 정수 비교를 **뺄셈**으로 처리하는 것도 오버플로 위험이 있어 피해야 한다.

```
return o1.getValue() - o2.getValue(); // 위험  
return Integer.compare(o1.getValue(), o2.getValue()); // 안전
```

2. Comparator 가 equals 와 불일치하는 경우

`Comparator` 가 `equals` 와 불일치하면 `TreeSet`, `TreeMap` 같은 정렬 컬렉션에서 치명적이다. 정렬 컬렉션은 동등성 판단에 `equals` 가 아니라 비교 결과(0인지 여부)를 사용하기 때문이다.

같은 입사일을 가진 두 직원이 동등하다고 판정되면, 두 번째 직원은 중복으로 간주되어 무시된다. 이를 해결하려면 객체를 유일하게 식별하는 속성을 비교 기준에 함께 포함시킨다.

```
Comparator<Employee> safe = Comparator.comparing(Employee::hireDate)  
                                        .thenComparing(Employee::id); // 유일  
식별자로 tie-break
```

2.3 The SortedSet Interface

`SortedSet` 은 원소를 오름차순으로 유지하는 `Set` 으로, **자연 순서 또는 생성 시 제공된 `Comparator` 에 따라 정렬된다.** `Set` 연산에 더해 **범위 뷰, 끝점, `Comparator` 접근**의 세 부류를 추가한다.

```
public interface SortedSet<E> extends Set<E>, SequencedSet<E> {

    Comparator<? super E> comparator();

    SortedSet<E> subSet(E fromElement, E toElement);

    SortedSet<E> headSet(E toElement);

    SortedSet<E> tailSet(E fromElement);

    E first();

    E last();

    // 이하 생략
}
```

`SequencedSet` 은 Java 21에서 도입된 인터페이스로, 순서가 정의된 컬렉션에 공통 API(`getFirst`, `reversed` 등)를 제공한다.

상속한 `Set` 연산은 두 가지 예외를 제외하면 동일하게 동작한다. `iterator` 가 **정렬 순서대로 순회**하고, `toArray` 가 **정렬된 순서**로 원소를 담는다.

범위 뷰

리스트의 범위 뷰와 결정적인 차이가 있다. **정렬 집합의 범위 뷰는 집합이 직접 수정되어도 계속 유효하게 유지된다.** 끝점이 특정 원소(인덱스)가 아니라 **원소 공간 상의 절대적인 기점**이기 때문이다. 따라서 리스트의 `subList` 와 달리 오래 사용해도 안전하다.

```
int count = dictionary.subSet("doorbell", "pickle").size();
dictionary.subSet("f", "g").clear(); // f 로 시작하는 원소 전부 제거
```

`subSet` 의 구간은 `[fromElement, toElement)` 로 **시작은 포함, 끝은 제외**이다. 닫힌 구간을 보려면 **후속자(successor)** 를 이용하는데, `String` 자연 순서에서 문자열 `S` 의 후속자는 `S + "\0"` 이다.

```
// low 이상 high 이하 (양쪽 모두 포함)
SortedSet<String> closed = dictionary.subSet(low, high + "\0");
```

`headSet` 은 지정 원소 **앞까지의** 앞부분 뷰를, `tailSet` 은 지정 원소 **부터** 끝까지의 뒷부분 뷰를 반환한다.

끝점과 Comparator 접근

끝점 연산 `first`, `last` 도 제공된다. 이는 `SortedSet` 이 역방향 반복자를 제공하지 않는 한계를 우회하는 데에도 쓰이는데, 내부에서 역방향으로 진행하려면 `ss.headSet(o).last()` 로 한 원소 뒤로 갈 수 있다. 다만 이를 반복하는 것은 원소마다 조회가 필요해 비효율적이다.

실무에서는 `SortedSet` 을 확장한 `NavigableSet` (`TreeSet` 이 구현)의 `descendingSet`, `descendingIterator`, `lower`, `higher`, `floor`, `ceiling` 을 사용하는 편이 훨씬 효율적이다.

`comparator` 접근자는 정렬에 사용된 `Comparator` 를 반환하며, 자연 순서로 정렬된 경우 `null` 을 반환한다.

2.4 The SortedMap Interface

`SortedMap` 은 항목을 **키의 오름차순**으로 유지하는 맵으로, `SortedSet` 의 맵 대응판이다.

```
public interface SortedMap<K, V> extends SequencedMap<K, V> {

    Comparator<? super K> comparator();

    SortedMap<K, V> subMap(K fromKey, K toKey);

    SortedMap<K, V> headMap(K toKey);

    SortedMap<K, V> tailMap(K fromKey);

    K firstKey();

    K lastKey();

    Set<K> keySet();

    Collection<V> values();

    Set<Map.Entry<K, V>> entrySet();

    V putFirst(K k, V v);

    V putLast(K k, V v);

    SortedMap<K, V> reversed();
}
```

`putFirst`, `putLast` 는 `SequencedMap` 에서 상속된 연산이지만, `SortedMap` 은 키 순서가 자동으로 결정되므로 **항상** `UnsupportedOperationException` 을 던진다.

상속한 맵의 연산은 컬렉션 뷰의 `iterator` 가 키의 정렬 순서대로 순회하고, `toArray` 가 정렬된 순서로 담는다는 두 예외를 제외하면 동일하게 동작한다.

범위 뷰(`subMap` , `headMap` , `tailMap`), 끝점(`firstKey` , `lastKey`), `comparator` 접근자의 의미는 `SortedSet` 과 동일하다.

```
SortedMap<String, Integer> scores = new TreeMap<>();
scores.put("banana", 2);
scores.put("apple", 1);
scores.put("cherry", 3);

System.out.println(scores);           // {apple=1, banana=2, cherry=3}
System.out.println(scores.firstKey()); // apple
System.out.println(scores.headMap("c")); // {apple=1, banana=2}
```
